14 Introduction to Posterior Simulation

- All of Bayesian inference is based on the posterior distribution of parameters given sample data.
- Therefore, the main computational task in a Bayesian analysis is computing, or approximating, the posterior distribution.

Example 14.1. Suppose we wish to estimate the proportion of Cal Poly students that are left-handed. Suppose we want to assume a Normal distribution prior for θ with mean 0.15 and SD 0.08. Also suppose that in a sample of 25 Cal Poly students 5 are left-handed. We want to find the posterior distribution.

Note: the Normal distribution prior assigns positive (but small) density outside of (0, 1). So we can either truncate the prior to 0 outside of (0, 1) or just rely on the fact that the likelihood will be 0 for θ outside of (0, 1) to assign 0 posterior density outside (0, 1).

1. Write an expression for the shape of the posterior density. Is this a recognizable probability distribution?

2. Describe in full detail two methods for approximating the posterior distribution in this situation.

- In most problems it is not possible to find the posterior distribution analytically, and therefore we must approximate it.
- We have seen two approximation methods: grid approximation and simulation.

Example 14.2. Suppose you're interested in studying sleep hours of Cal Poly undergraduates. You assume that within each of the six colleges sleep hours follow a Normal distribution with a

mean and SD for the college. You'll collect data on a random sample of students, record their college and sleep hours, and use it to perform a Bayesian analysis.

1. How many parameters are there in this situation?

2. Suppose you try a grid approximation. You create a grid of 100 values over the possible range of values of each parameter. How many rows would there be in the Bayes table?

- Grid approximation suffers from the "curse of dimensionality" and is not computationally feasible in multi-parameter problems (and most interesting problems are multi-parameter).

Example 14.3. Recall that we have used simulation to approximate posterior distributions in a few situations: Example 2.3, Example 10.2, Example 10.7.

1. Explain the general method for using simulation to approximate a posterior distribution given sample data. Careful: don't confuse posterior distributions with posterior predictive distributions.

2. Explain why this method is typically not feasible in practice. (It might help to look at the simulation results from the examples referenced above.)

- We have seen three ways to compute/approximate the posterior distribution
  - *Math.* Only works in a few special cases (e.g., Beta-Binomial, Normal-Normal, conjugate prior situations).

  - *Grid approximation.* Not feasible when there are more than just a few parameters.
  - *Simulation.* By far the most common method, though we have only seen a very naive approach so far.

- In principle, the posterior distribution $\pi(\theta|y)$ of parameters θ given observed data y can be approximated via simulation in the following way.
  - Simulate a θ value from the prior distribution (θ could be a vector of parameters, e.g., $\theta = (\mu, \sigma)$).
  - Given the simulated θ, simulate hypothetical data y from the corresponding conditional distribution of y given θ (y could be a full sample, or the value of a sample statistic).
  - Repeat many times and discard (θ, y) pairs for which the simulated data y is not equal to the observed data y
  - Summarize the simulated θ values for the remaining pairs (with y equal to the observed data).

- While this method works in principle, it is a very computationally inefficient way of approximating the posterior distribution, since in most cases (e.g., not-small sample, continuous variables) the probability of replicating the observed data y is essentially 0.
- Therefore, we need more efficient simulation algorithms for approximating posterior distributions. **Markov chain Monte Carlo (MCMC)** methods provide powerful and widely applicable algorithms for simulating from probability distributions, including complex and high-dimensional distributions. These algorithms include Metropolis-Hastings, Gibbs sampling, Hamiltonian Monte Carlo (HMC), and No U-Turn sampling (NUTS), among others. We will see later some of the ideas behind MCMC algorithms.
- Regardless of which simulation algorithm we use to approximate the posterior distribution, there are 3 main inputs:
  - Observed data y
  - Model for the data, $f(y|\theta)$ which depends on parameters θ. (This model determines the likelihood function.)
  - Prior distribution for parameters $\pi(\theta)$

- We then employ some simulation algorithm to approximate the posterior distribution of θ given the observed data y, $\pi(\theta|y)$, *without computing the posterior distribution.*

14.1 Introduction to `brms`

- We will rely on the software package `brms` ("Bayesian Regression Models using Stan") to carry out MCMC simulations.
- `brms` is an interface for the MCMC software Stan.
- Here are a few references for using `brms`

- `brms` site
- *Bayesian Modeling using Stan* by Jim Albert (companion to Albert and Hu textbook)
- *Doing Bayesian Data Analysis in brms and the tidyverse* by A Solomon Kurz (companion to Kruschke's *Doing Bayesian Data Analysis*)
- Parameterization of Response Distributions in `brms`
- `brms` reference manual

```
library(brms)
```

```
Loading required package: Rcpp
```

```
Loading 'brms' package (version 2.23.0). Useful instructions
can be found by typing help('brms'). A more detailed introduction
to the package is available through vignette('brms_overview').
```

```
Attaching package: 'brms'
```

```
The following objects are masked from 'package:tidybayes':

    dstudent_t, pstudent_t, qstudent_t, rstudent_t
```

```
The following object is masked from 'package:stats':

    ar
```

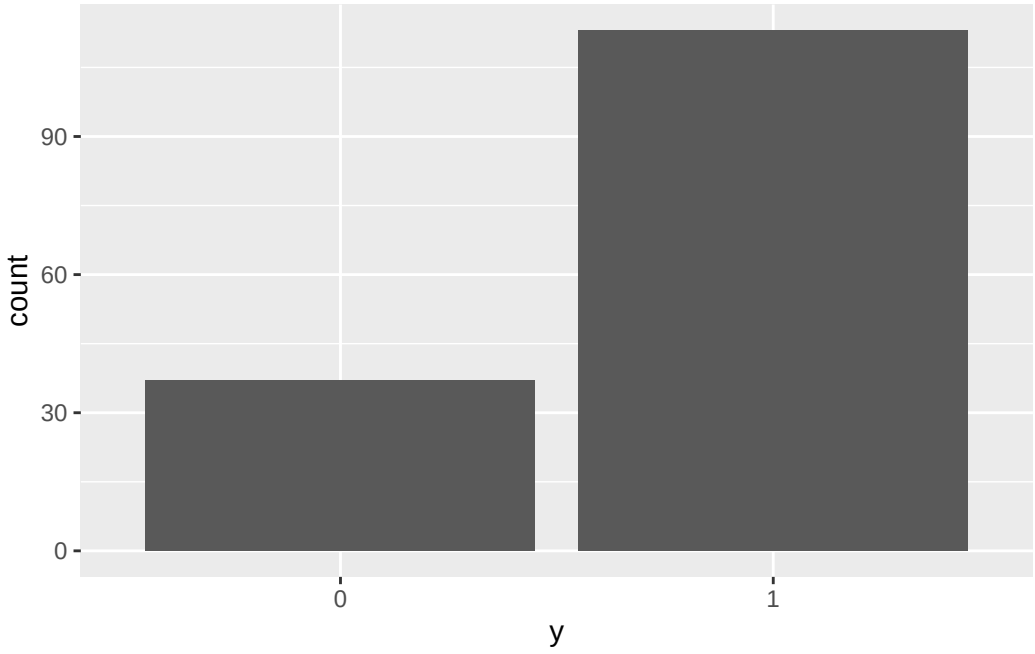
We will introduce some basic functionality of `brms` in a few examples we have encountered previously.

14.1.1 Example: inference for a population proportion

Recall Example 7.3 where we were interested in θ, the population proportion of American adults who have read a book in the last year. We assumed a Normal prior distribution for θ with prior mean 0.4 and prior standard deviation 0.1. In a sample of 150 American adults, 113 have read a book in the past year. (The real sample was size 1502, but we looked at the smaller sample size first.)

First we define the data. We could just define the total number of successes and the sample size. However, data is most often input as individual values in un-summarized form (e.g., a csv file with one row for each observational unit), so this is what we'll do. We'll create a vector with 113 1s ("successes") and 37 0s ("failures").

```r
data = data.frame(y = c(rep(1, 113), rep(0, 37)))

ggplot(data, aes(x = factor(y))) + geom_bar() + labs(x = "y")
```



Note that since the data contains individual 1/0 values, the likelihood is determined by the *Bernoulli* distribution, a special case of Binomial with $n = 1$. That is, if y_i denotes an individual value, then $y_i \sim \text{Bernoulli}(\theta) = \text{Binomial}(1, \theta)$.

Next we fit the model to the data using the function `brm`. The three inputs are:

- Input 1: the data
  `data = data`
- Input 2: model for the data (likelihood)
  `family = bernoulli(link = identity)`
  `y ~ 1 # linear model with intercept only`
- Input 3: prior distribution for parameters
  `prior = prior(normal(0.4, 0.1), class = Intercept)`
- Other arguments specify the details of the numerical algorithm; our specifications below will result in 10000 θ values simulated from the posterior distribution.

Note: as its name suggests `brms` is made for regression models (broadly). Therefore, we will usually specify the `formula` with regression model syntax. This will be more natural when we

actually get to regression models, but it's a little clunky for some of the very basic models we have seen so far.

```
fit <-
  brm(
    data = data,
    family = bernoulli(link = identity),
    formula = y ~ 1,
    prior(normal(0.4, 0.1), class = Intercept),
    iter = 11000,
    warmup = 1000,
    chains = 1,
    refresh = 0
  )
```

```
Compiling Stan program...
```

```
Start sampling
```

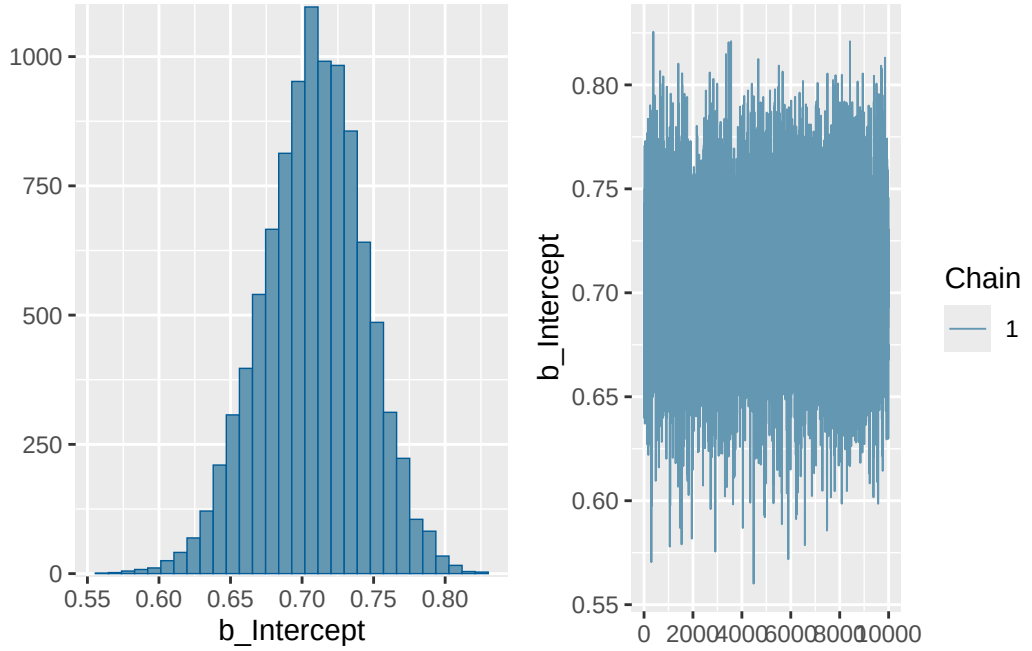
Here is a brief summary of results. Notice that "Estimate" is the posterior mean, " "Est.Error" is the posterior standard deviation, and "l-95% CI, u-95% CI" are the endpoints of a posterior 95% credible interval.

```
summary(fit)
```

```
 Family: bernoulli
  Links: mu = identity
Formula: y ~ 1
   Data: data (Number of observations: 150)
  Draws: 1 chains, each with iter = 11000; warmup = 1000; thin = 1;
         total post-warmup draws = 10000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     0.71      0.04     0.64     0.77 1.00     4009     5442

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

Here are some plots. The plot on the left represents the posterior distribution of θ.

```
plot(fit)
```



The `b_Intercept` row in the `posterior_summary` below contains the posterior mean ("Estimate"), SD ("Est.Error"), median ("Q50"), and endpoints of central 50%, 80%, and 98% credible intervals. Compare to the grid approximation in Example 7.3; the results are very similar.

```
posterior_summary(fit, probs = c(0.01, 0.10, 0.25, 0.50, 0.75, 0.90, 0.99))
```

```
             Estimate  Est.Error         Q1         Q10         Q25
b_Intercept   0.707897 0.03556536   0.620596   0.6612202   0.6848232
Intercept     0.707897 0.03556536   0.620596   0.6612202   0.6848232
lprior       -3.419621 1.08764709  -6.127092  -4.8261475  -4.1310129
lp__       -88.410800 0.70711336 -91.295426 -89.2551486 -88.5580201
                  Q50         Q75         Q90         Q99
b_Intercept   0.7090404   0.7321042   0.7524144   0.7875755
Intercept     0.7090404   0.7321042   0.7524144   0.7875755
lprior       -3.3916527  -2.6725663  -2.0281518  -1.0494835
lp__       -88.1351184 -87.9695356 -87.9313261 -87.9239804
```

.chain	.iteration	.draw	theta
1	1	1	0.7166199
1	2	2	0.7324163
1	3	3	0.6957477
1	4	4	0.7426208
1	5	5	0.7083129
1	6	6	0.6874246
1	7	7	0.6792586
1	8	8	0.6398941
1	9	9	0.6795586
1	10	10	0.6714900

There are commands within `brms` for working with simulation output. But using `spread_draws` from the `tidybayes` package, we can put the 10000 simulated values into a data frame. Then we can summarize the posterior distribution just like we summarize any data set.

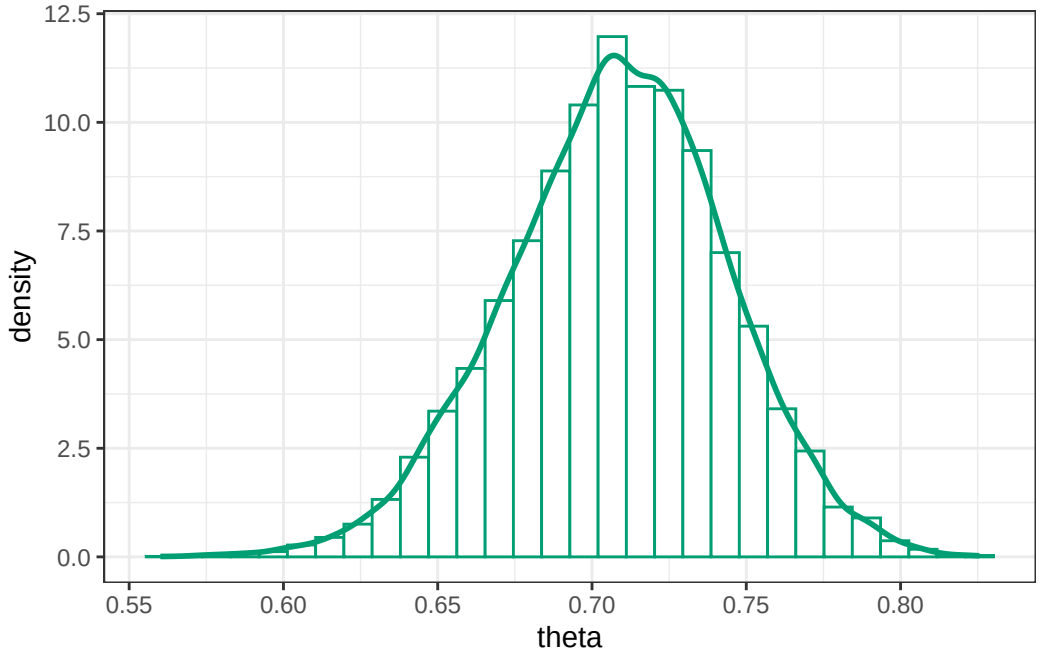
Below we save simulated values of `theta` in the `posterior` data frame.

```
library(tidybayes)

posterior_df = fit |>
  spread_draws(b_Intercept) |>
  rename(theta = b_Intercept)

posterior_df |> head(10) |> kbl() |> kable_styling()
```

Now we can plot values of `theta` as usual, e.g., using `ggplot2`.

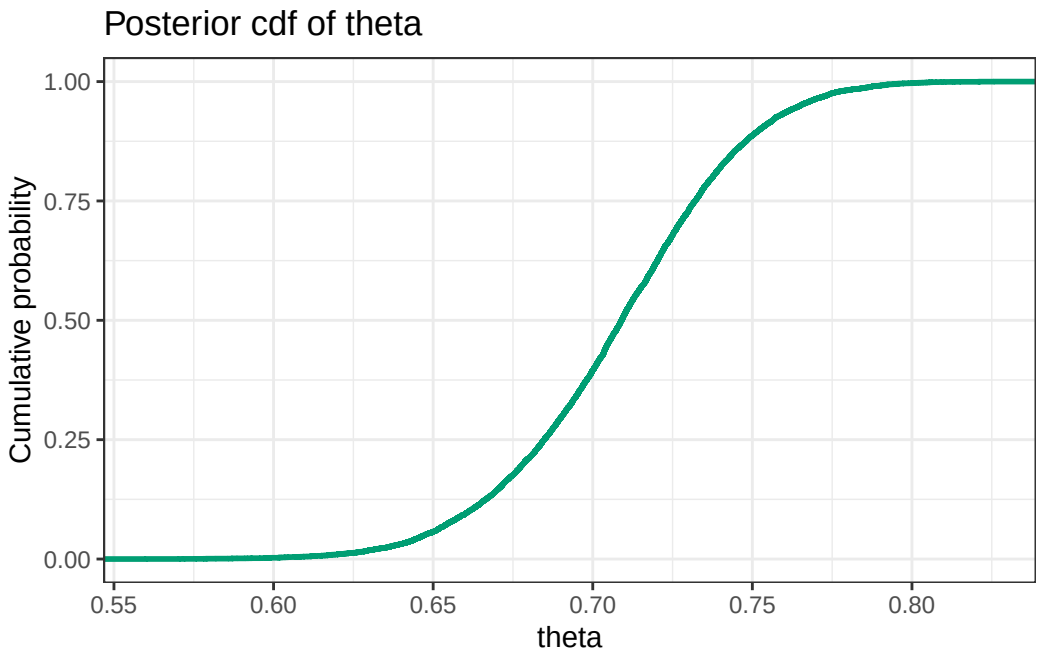
Here is a histogram of the simulated θ values, with the approximate posterior density overlaid.

```
posterior_df |>
  ggplot(aes(x = theta)) +
  geom_histogram(
    aes(y = after_stat(density)),
    col = bayes_col["posterior"],
    fill = "white"
  ) +
  geom_density(linewidth = 1, col = bayes_col["posterior"]) +
  theme_bw()
```



Here is a plot of the approximate posterior cumulative distribution function.

```
posterior_df |>
  ggplot(aes(x = theta)) +
  stat_ecdf(col = bayes_col["posterior"], linewidth = 1) +
  labs(title = "Posterior cdf of theta", y = "Cumulative probability") +
  theme_bw()
```



We can use the simulated values to approximate posterior mean and SD as usual.

```
mean(posterior_df$theta)
```

```
[1] 0.707897
```

```
sd(posterior_df$theta)
```

```
[1] 0.03556536
```

Quantiles of the posterior distribution of θ are endpoints of credible intervals.

```
quantile(posterior_df$theta, c(0.01, 0.10, 0.25, 0.75, 0.90, 0.99))
```

```
       1%       10%       25%       75%       90%       99%
0.6205960 0.6612202 0.6848232 0.7321042 0.7524144 0.7875755
```

We can approximate the posterior probability that θ is greater than 0.7 as usual.

```
sum(posterior_df$theta > 0.7) / length(posterior_df$theta)
```

```
[1] 0.6042
```

```
library(posterior)
```

```
This is posterior version 1.6.1


Attaching package: 'posterior'

The following objects are masked from 'package:stats':

    mad, sd, var

The following objects are masked from 'package:base':

    %in%, match
```

The function `as_draws_df` from the `posterior` packages also puts the `brms` output into a data frame (similar to `tidybayes::spread_draws`).

```
post_draws <- as_draws_df(fit)

head(post_draws)
```

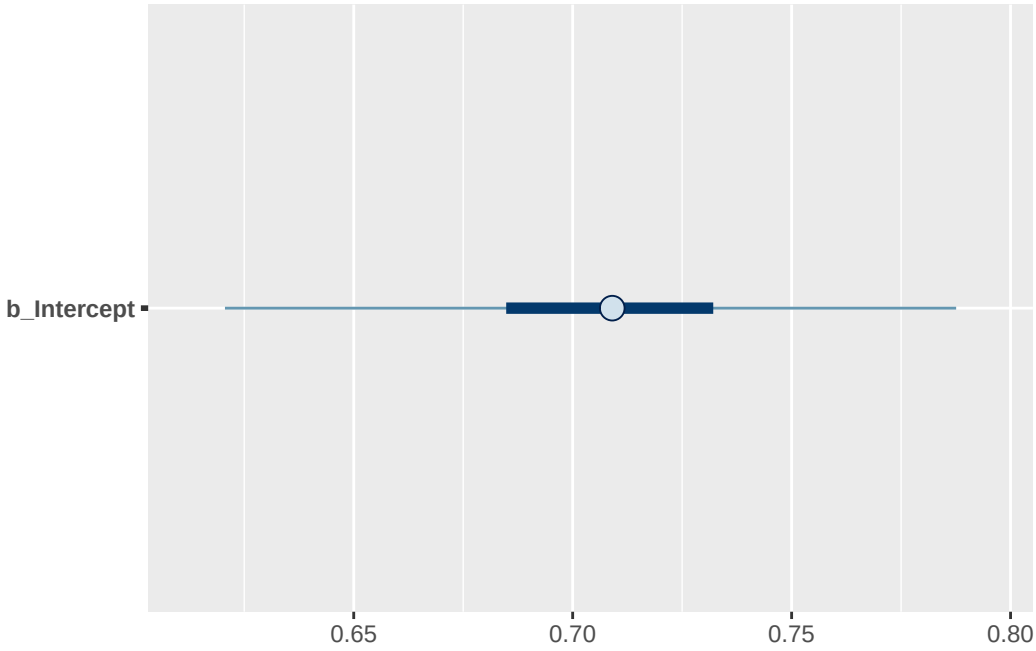
```
# A draws_df: 6 iterations, 1 chains, and 4 variables
  b_Intercept Intercept lprior lp__
1        0.72      0.72   -3.6  -88
2        0.73      0.73   -4.1  -88
3        0.70      0.70   -3.0  -88
4        0.74      0.74   -4.5  -88
5        0.71      0.71   -3.4  -88
6        0.69      0.69   -2.7  -88
# ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

The `bayesplot` package has lots of nice built in plots.

```
library(bayesplot)
```

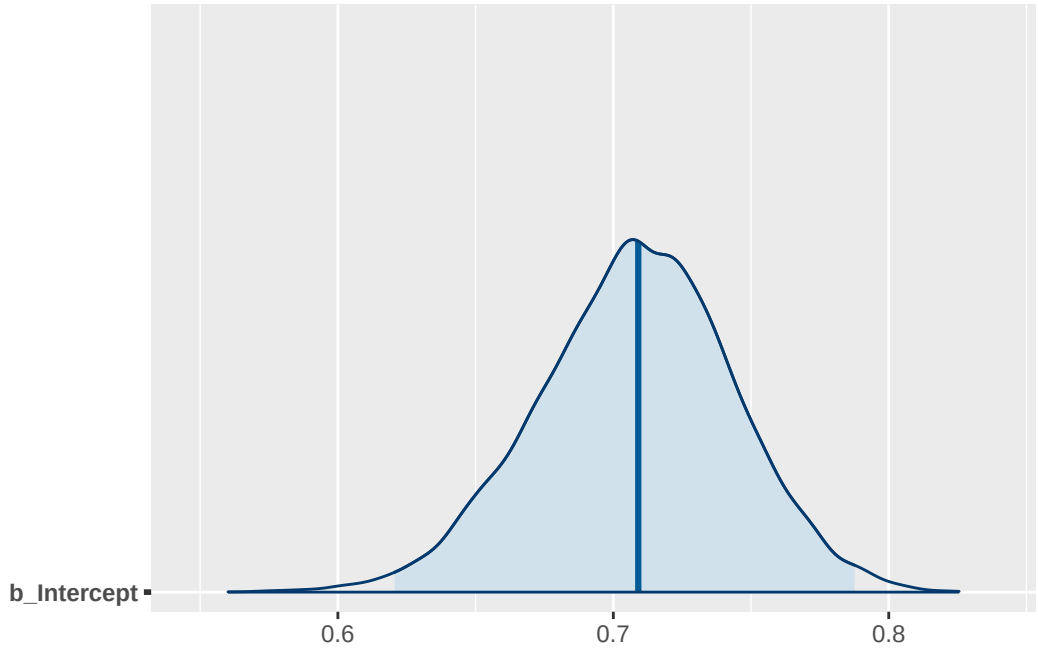
```
This is bayesplot version 1.14.0

- Online documentation and vignettes at mc-stan.org/bayesplot

- bayesplot theme set to bayesplot::theme_default()

   * Does _not_ affect other ggplot2 plots

   * See ?bayesplot_theme_set for details on theme setting


Attaching package: 'bayesplot'

The following object is masked from 'package:posterior':

    rhat

The following object is masked from 'package:brms':

    rhat
```

```
mcmc_intervals(
  post_draws,
  pars = "b_Intercept",
  prob = 0.50,
  prob_outer = 0.98
)
```



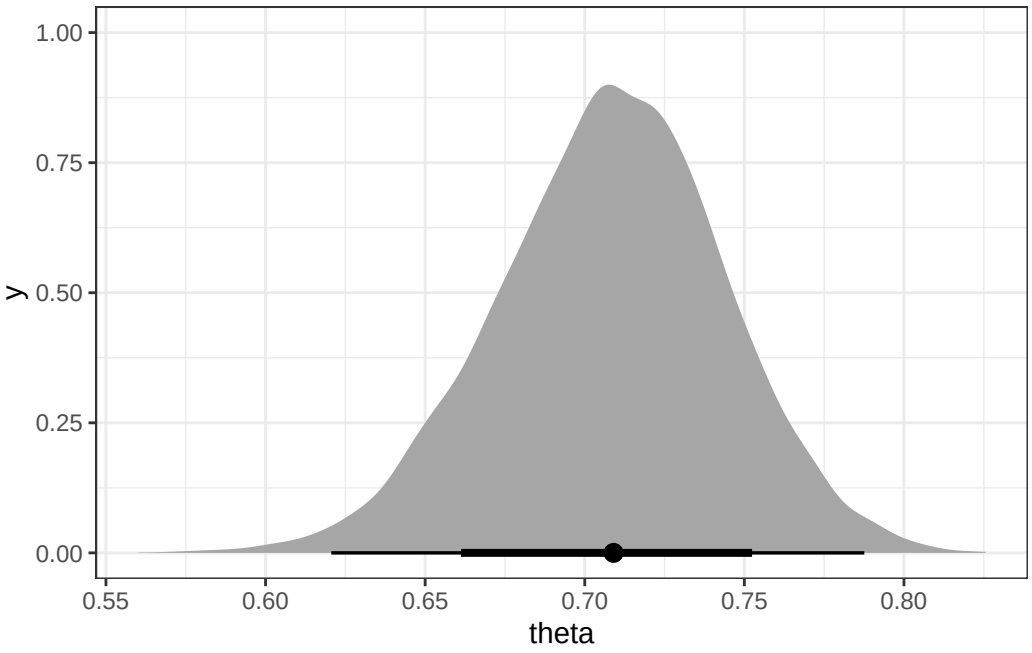
It is often helpful to create a data frame for the simulated values of the parameter, but it's not necessary for all functions. For example, we can just pass the `brms` output `fit` into `mcmc_areas`.

```
mcmc_areas(
  fit,
  pars = "b_Intercept",
  prob = 0.98
)
```



The `ggdist` package includes some add-ons for `ggplot2` that are useful for visualizing posterior distributions.

```
posterior_df |>
  ggplot(aes(x = theta)) +
  stat_halfeye(.width = c(0.80, 0.98)) +
  theme_bw()
```



14.1.2 Example: inference for a population mean

Now we'll look at an example with continuous data. Recall Example 8.4. Assume body temperatures (degrees Fahrenheit) of healthy adults follow a Normal distribution with unknown mean θ and known standard deviation $\sigma = 1.1$. (It's unrealistic to assume the population standard deviation is known. We'll consider the case of unknown standard deviation later.) Suppose we wish to estimate θ, the population mean healthy human body temperature. We considered a few priors in Example 8.4; here we'll use a Uniform prior on [96.0, 100.0]. In a recent study, the sample mean body temperature in a sample of 208 healthy adults was 97.7 degrees F.

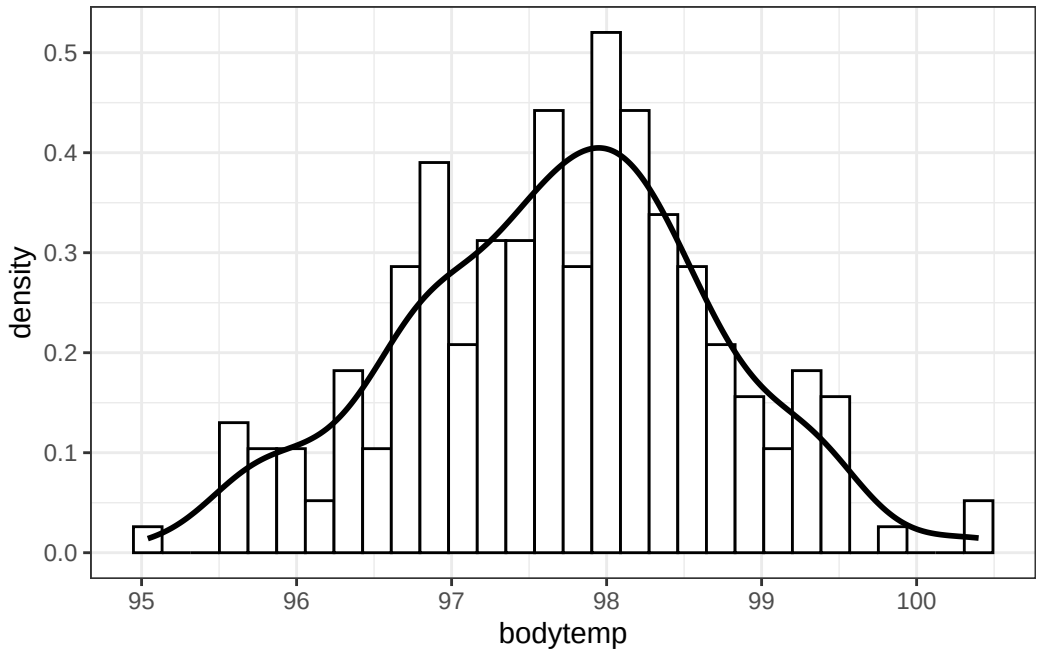
First we load the data from the file `bodytemps.csv`; the temperature variable is `bodytemp`.

```
data = read_csv("bodytemps.csv")

head(data)
```

```
# A tibble: 6 x 1
  bodytemp
     <dbl>
1     95.6
2     97.3
```

```
3     98.0
4     97.0
5     96.0
6     98.6
```

```
data |>
  ggplot(aes(x = bodytemp)) +
  geom_histogram(aes(y = after_stat(density)), col = "black", fill = "white") +
  geom_density(linewidth = 1) +
  theme_bw()
```



Next we fit the model to the data using the function `brm`. The three inputs are

- Input 1: the data
  `data = data`
- Input 2: model for the data (likelihood)
  `family = gaussian`
  `bodytemp ~ 1 # linear model with intercept (mean) only`
- Input 3: prior distribution for parameters
  `prior = c(prior(uniform(96, 100), lb = 96, ub = 100, class = Intercept),`
  `prior(constant(1.1), class = sigma))`

Note that the Normal (`gaussian`) model for the likelihood assumes two parameters - mean ("intercept") and SD ("sigma") - so we need to specify a prior distribution for both parameters. Since we're assuming that SD σ is known, we fix its value to be constant.

- Other arguments specify the details of the numerical algorithm; our specifications below will result in 10000 (μ, σ) pairs simulated from the posterior distribution; a few simulated values are displayed below.

```
fit <- brm(
  data = data,
  family = gaussian,
  bodytemp ~ 1,
  prior = c(
    prior(uniform(96, 100), lb = 96, ub = 100, class = Intercept),
    prior(constant(1.1), class = sigma)
  ),
  iter = 11000,
  warmup = 1000,
  chains = 1,
  refresh = 0
)
```

```
Compiling Stan program...
```

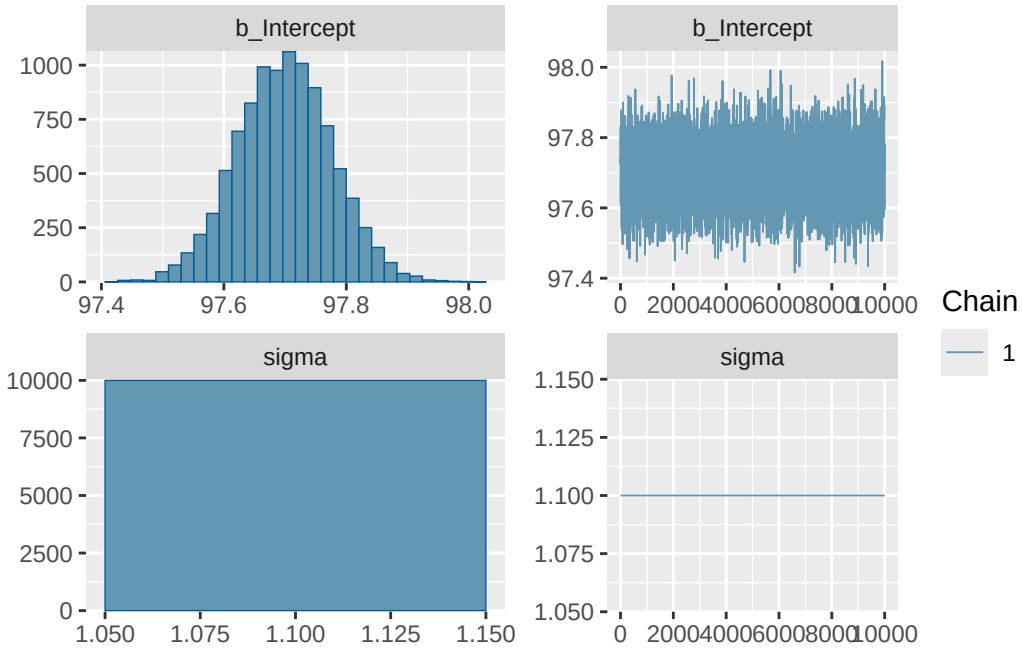
```
Start sampling
```

```
summary(fit)
```

```
 Family: gaussian
  Links: mu = identity
Formula: bodytemp ~ 1
   Data: data (Number of observations: 208)
  Draws: 1 chains, each with iter = 11000; warmup = 1000; thin = 1;
         total post-warmup draws = 10000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept    97.70      0.08    97.55    97.85 1.00     3771     6432

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
```

```
sigma     1.10      0.00     1.10     1.10    NA       NA       NA

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(fit)
```



Below we save simulated values of `theta` in the `posterior` data frame.

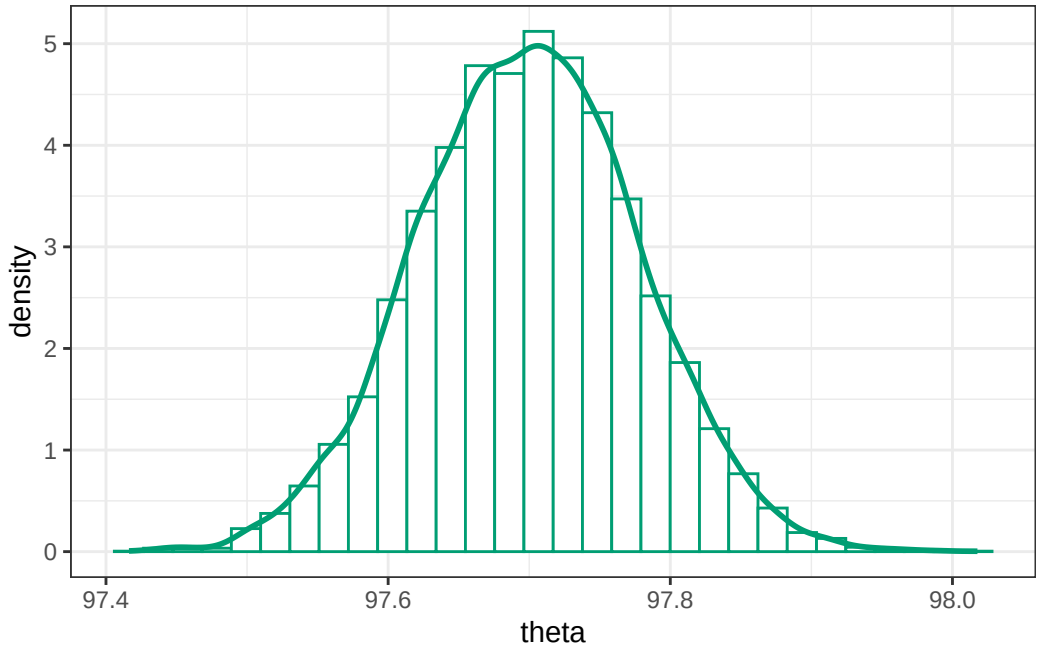
```
posterior_df = fit |>
  spread_draws(b_Intercept) |>
  rename(theta = b_Intercept)

posterior_df |> head(10) |> kbl() |> kable_styling()
```

Here is a histogram of the simulated θ values, with the approximate posterior density overlaid.

```
posterior_df |>
  ggplot(aes(x = theta)) +
  geom_histogram(
    aes(y = after_stat(density)),
```

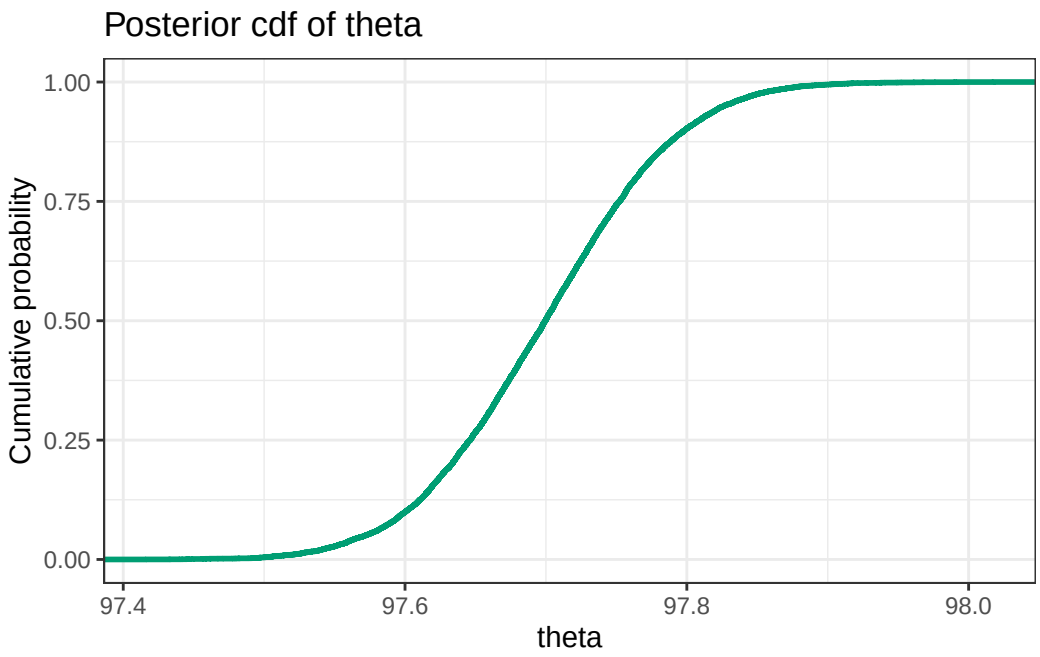
.chain	.iteration	.draw	theta
1	1	1	97.72510
1	2	2	97.72510
1	3	3	97.74661
1	4	4	97.83249
1	5	5	97.65610
1	6	6	97.61270
1	7	7	97.65778
1	8	8	97.66590
1	9	9	97.68526
1	10	10	97.68531

```
  col = bayes_col["posterior"],
  fill = "white"
) +
geom_density(linewidth = 1, col = bayes_col["posterior"]) +
theme_bw()
```



Here is a plot of the approximate posterior cumulative distribution function.

```
posterior_df |>
  ggplot(aes(x = theta)) +
  stat_ecdf(col = bayes_col["posterior"], linewidth = 1) +
  labs(title = "Posterior cdf of theta", y = "Cumulative probability") +
  theme_bw()
```



We can use the simulated values to approximate posterior mean and SD as usual.

```
mean(posterior_df$theta)
```

```
[1] 97.69915
```

```
sd(posterior_df$theta)
```

```
[1] 0.07771218
```

Quantiles of the posterior distribution of θ are endpoints of credible intervals.

```
quantile(posterior_df$theta, c(0.01, 0.10, 0.25, 0.75, 0.90, 0.99))
```

```
      1%      10%      25%      75%      90%      99%
97.51908 97.60045 97.64632 97.75217 97.79897 97.87782
```

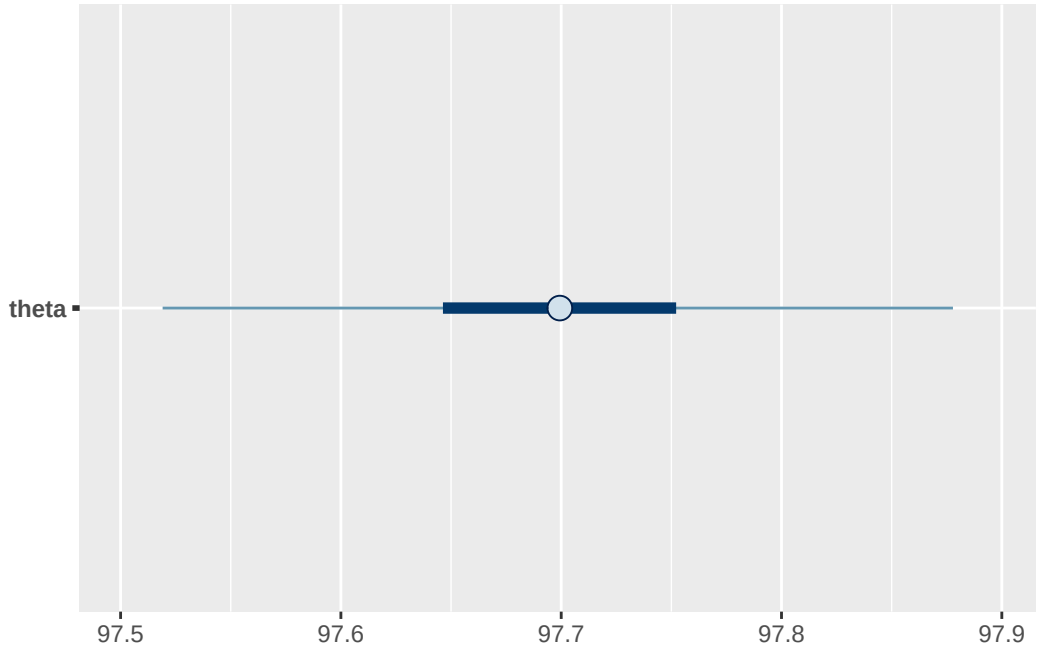
We can approximate the posterior probability that θ is less than 97.8.

```
sum(posterior_df$theta < 97.8) / length(posterior_df$theta)
```

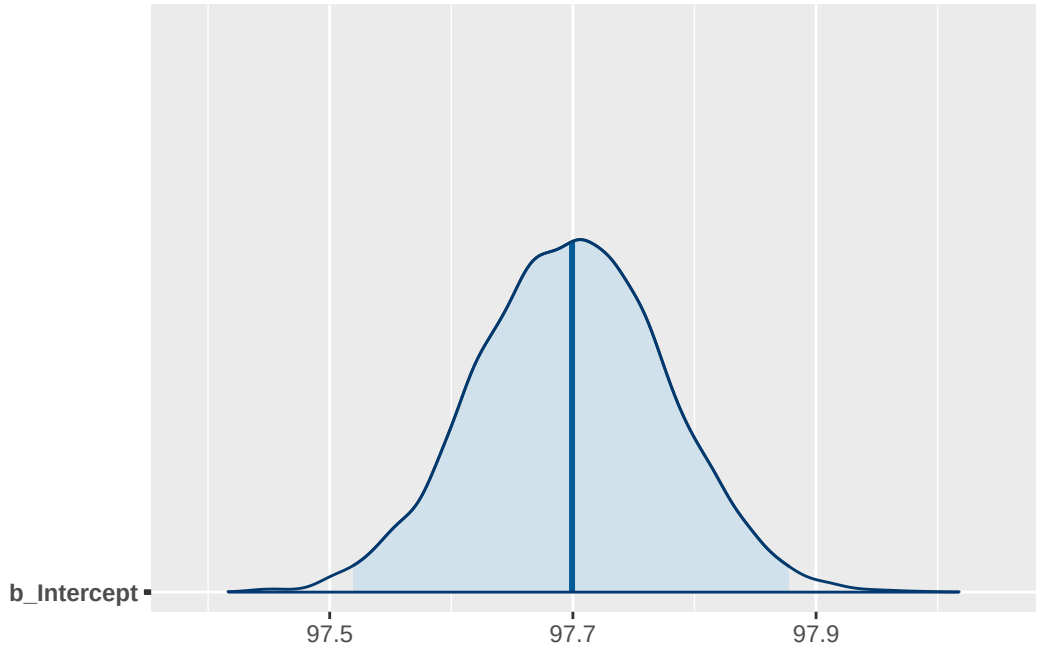
```
[1] 0.9028
```

A few more summaries.

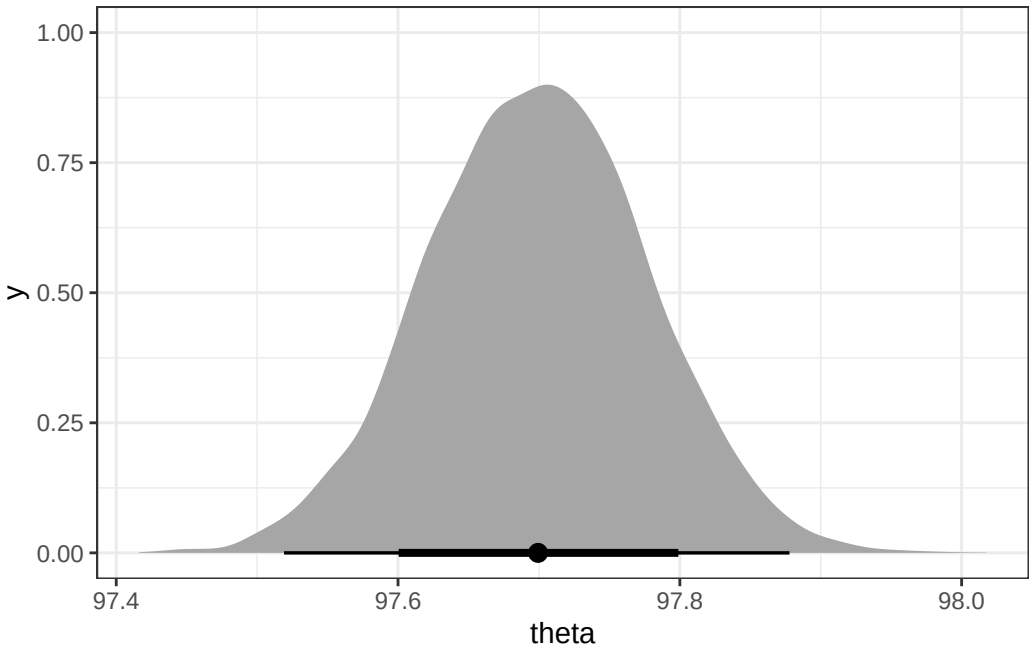
```
mcmc_intervals(
  posterior_df,
  pars = "theta",
  prob = 0.50,
  prob_outer = 0.98
)
```




```
mcmc_areas(
  fit,
  pars = "b_Intercept",
  prob = 0.98
)
```



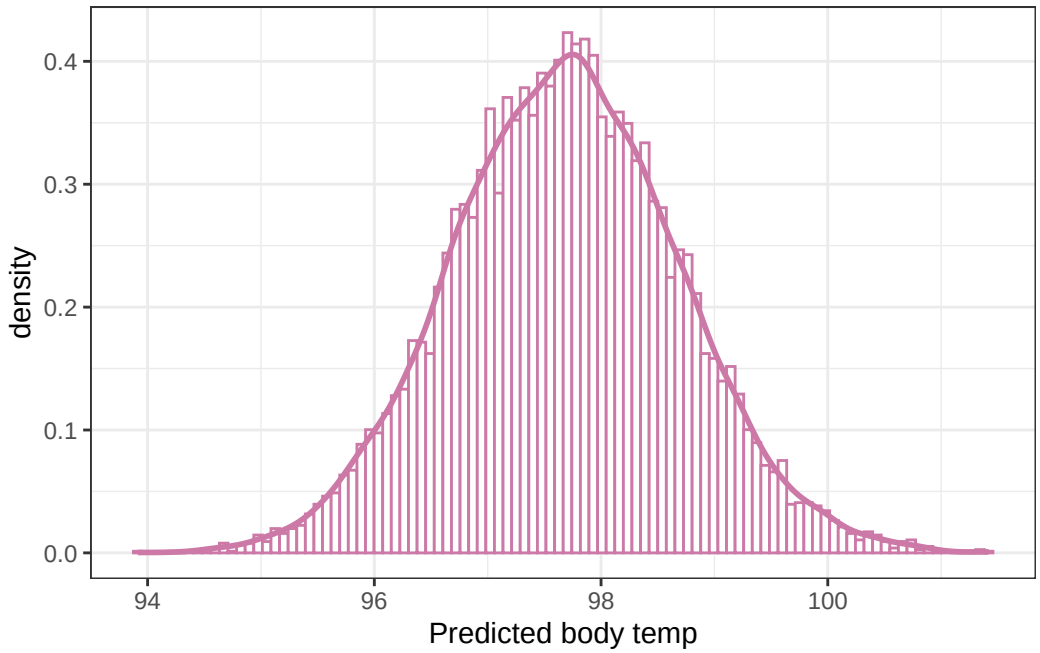

```
posterior_df |>
  ggplot(aes(x = theta)) +
  stat_halfeye(.width = c(0.80, 0.98)) +
  theme_bw()
```



14.1.3 Example: Posterior predictive distribution and checking

Continuing the body temperature example, we can use the simulated values of θ to approximate the posterior predictive distribution. For each simulated value of θ, simulate a y value from a Normal(θ, 1) distribution, and summarize the simulate y values.

```
n_rep = length(posterior_df$theta)

y_sim = rnorm(n_rep, posterior_df$theta, sd = 1)

ggplot(data.frame(y_sim), aes(x = y_sim)) +
  geom_histogram(
    aes(y = after_stat(density)),
    col = bayes_col["posterior_predict"],
    fill = "white",
    bins = 100
  ) +
  geom_density(linewidth = 1, col = bayes_col["posterior_predict"]) +
  labs(x = "Predicted body temp") +
  theme_bw()
```



Quantiles of the posterior predictive distribution of y are endpoints of posterior prediction intervals.

```
quantile(posterior_df$theta, c(0.025, 0.975))
```

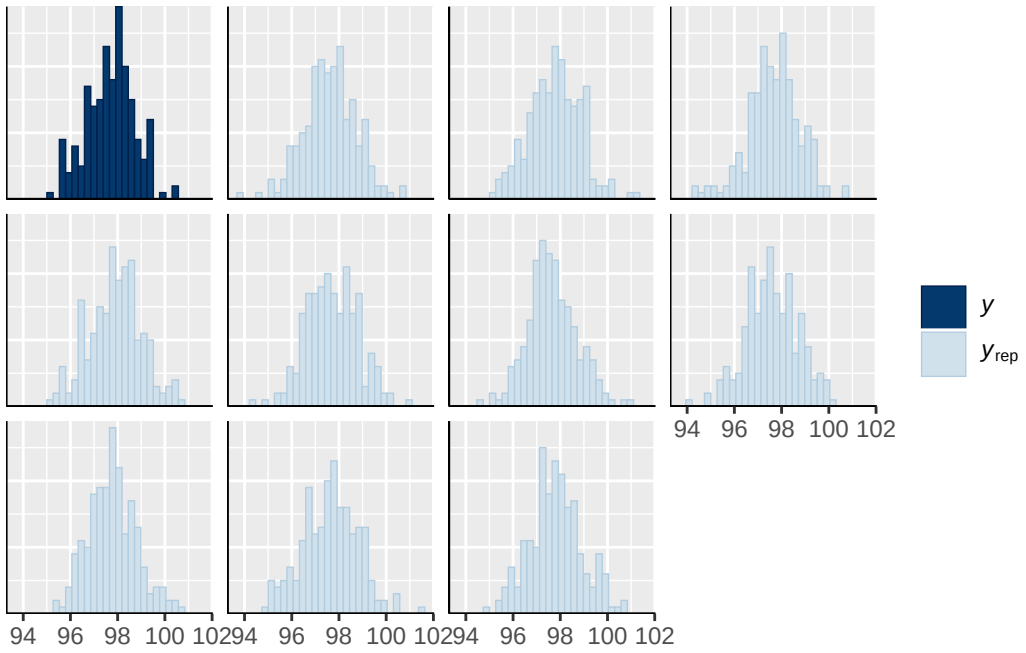
```
    2.5%    97.5% 
97.54647 97.85055 
```

We can use `pp_check` to perform a posterior predictive check: simulate hypothetical samples of size 208 according to the model, and compare to the observed data. The plot below contains a histogram of predicted y values for each of the simulated samples, along with a histogram of the observed data.

```
pp_check(fit, type = "hist")
```

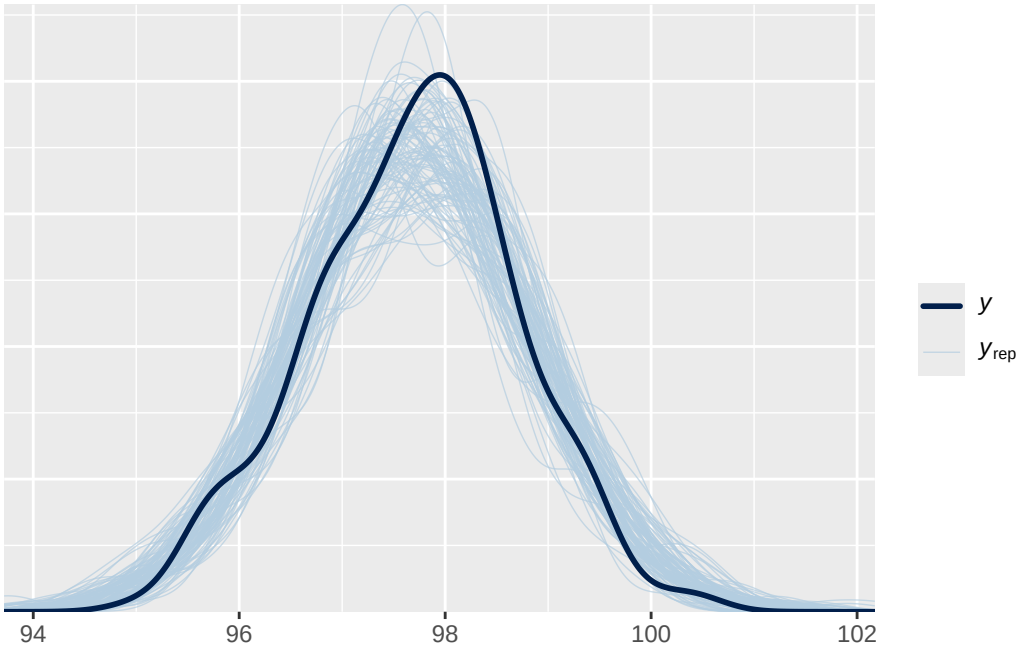
```
Using 10 posterior draws for ppc type 'hist' by default.
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



The default plot output of `pp_check` displays the density of each of the simulated samples overlaid on the estimated density of the observed sample data. Note that `ndraws` is the number of simulated *samples*, each sample of size 208.

```
pp_check(fit, ndraw = 100)
```


94
96
98
100
102
y
y rep